B.Sc. Engg. SWE 1st Semester

06 April 2026 (Afternoon)

# ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)

## ORGANISATION OF ISLAMIC COOPERATION (OIC)

## Department of Computer Science and Engineering (CSE)

SEMESTER FINAL EXAMINATION

WINTER SEMESTER, 2024-2025

DURATION: 2 HOURS

FULL MARKS: 120

# CSE 4107: Structured Programming I

**Programmable calculators are not allowed. Do not write anything on the question paper.**

Answer **all 4 (four)** questions. Figures in the right margin indicate full marks of questions with corresponding COs and POs in parentheses.

---

1. Suppose a hospital wants to maintain a database of patient records. Each patient will have the following four common attributes: `patientID` (integer), `name` (at most 30 characters), `age` (float), and `type` (value is 0 for indoor, and 1 for outdoor). Moreover, each patient will also have some special properties based on their type. Indoor patients will have two special attributes: `num_days_admitted` (integer), and `daily_charge` (double). Outdoor patients will have only one special attribute: `consultation_fee` (double). Note that a patient cannot be both indoor patient and outdoor patient at the same time.

   a) Based on the requirements, provide justifications regarding why you need to use unions in conjunction with structures for storing the data efficiently. — 6 (CO2) (PO2)

   b) Suppose the hospital is maintaining a file named *"patients.txt"* containing 100 such patient records. Assume that the file contains the information of each patient in separate lines, and each line contains the values in the same order mentioned in the scenario. Note that some lines will contain 6 values, which is the case if the $4^{th}$ value is 0 (indoor patient), and other lines will contain 5 values, which is the case if the $4^{th}$ value is 1 (outdoor patient). Write a C program that will take the input from the file and print the data on screen. — 12 (CO3) (PO1)

   c) Write a function `double outdoorFees(struct Patient patients[], int num_Patients)` that will take the array of patient records and the number of patients as parameter and return the sum of consultation fees of all outdoor patients. — 12 (CO3) (PO1)

2. Suppose you are working on developing an authentication system for your software, which will allow users to sign up with a unique username and set their own password. When setting the password for the first time, you want to enforce some restrictions so that users are not able to set passwords that will put their accounts at security risks. The password restrictions that you want to place are as follows:

   - Length should be at least 8 characters
   - Should contain at least one lowercase character, one uppercase character, and one digit
   - Same character cannot appear more than 3 times anywhere in the password

   a) Write a function `int password_validate(char *pass)` that takes a single string as parameter and returns 1 if it follows all the restrictions, and 0 if it fails to meet any one of the restrictions. — 16 (CO3) (PO1)

   b) You realized that even if the password restrictions are placed, the security risk will still remain if the password is simply stored in plaintext. So you decided to design a mechanism for hashing the password storing it in encrypted form. According to your hashing rule, the following conditions apply: — 14 (CO3) (PO1)

- All lowercase characters will convert to uppercase and shift by +2 positions (wrap around if needed), for example, `y` will become `A`
- All uppercase characters will convert to lowercase and shift by −3 positions (wrap around if needed), for example, `B` will become `y`
- All digits will convert to a new digit obtained as $(digit + 5)\%10$, for example, `9` will become `4`

Based on these rules, write a function `char* hashPassword(char *pass)` that takes a single string as parameter and returns a null-terminated hashed string.

3. Code Snippet 1 shows a code where a programmer attempted to process an array using nested loops. The program was intended to work as follows:

   - For every element `a[i]`, add all elements to its right with `a[i]`, if `a[i]` is even.
   - For every element `a[i]`, subtract all elements to its right from `a[i]`, if `a[i]` is odd.

   Unfortunately, the given code does not work as intended since there are some logical errors in the code.

```
1  #include <stdio.h>
2
3  int main() {
4      int arr[6] = {2, 5, 3, 7, 1, 4};
5      int i, j;
6
7      for (i = 0; i < 6; i++) {
8          for (j = i + 1; j < 6; j++) {
9              if (arr[i] % 2 == 0)
10                 arr[i] = arr[i] + arr[j];
11             else
12                 arr[i] = arr[i] - arr[j];
13         }
14     }
15
16     for (i = 0; i < 6; i++) {
17         printf("%d ", arr[i]);
18     }
19
20     return 0;
21 }
```

**Code Snippet 1**: An erroneous code for processing array using nested loop for Question 3

a) Determine the output of the faulty code given in Code Snippet 1. Show the state of the array after each outer loop iteration (every time value of `i` changes). 14 (CO1) (PO1)

b) Based on the original intention of the programmer, the expected output is `[22 -10 -9 2 -3 4]`. Identify the logical errors in the code given in Question 3 for which the expected output is not being produced. Specify the line number and explain why it is incorrect. Finally, rewrite the corrected version of the program so that it produces the expected output. 8+8 (CO2) (PO2)

4. a) Provide a comparative analysis of text file and binary file handling in C. 6 (CO2) (PO2)

b) Suppose an array of 4 integers given as `arr = [10, 200, -3, 45]` is written to a text file named *"data.txt"* and a binary file named *"data.bin"* as shown in Code Snippet 2. Determine the size (in bytes) of these two files. Also, write sample codes showing how the data can be read into two different arrays from these two files. 4 + 6 (CO1) (PO1)

```c
#include <stdio.h>

int main() {
    FILE *ft, *fb;
    int data[] = {10, 200, -3, 45};
    int i, value;

    ft = fopen("data.txt", "w");


    for (i = 0; i < 4; i++) {
        fprintf(ft, "%d ", data[i]);
    }
    fclose(ft);

    fb = fopen("data.bin", "wb");

    fwrite(data, sizeof(int), 4, fb);
    fclose(fb);
}
```

**Code Snippet 2:** Sample code for storing data in text file and binary file for Question 4.b

c) Suppose you are given a file named *"sorted_array.txt"* which contains $N, (0 \leq N \leq 1000)$ space-separated integers sorted in ascending order. Write a C program that takes a target value as input from the user and then reads data from the *"sorted_array.txt"* file and prints all possible pairs of elements whose sum exactly equals the target value. For this problem, you must use pointer arithmetic, and are restricted from using array subscripts. 14 (CO3) (PO1)

